

EXPERIMENT - 28

1. Problem Statement

Program 1. Write a C program to:

- Read the number of rows and columns from the user. Ensure that both values are positive. If not, repeat to get the desired input(s).
- Declare a variable-length 2D integer array using the given dimensions.
- Read the array elements column by column using nested loops, with the outer loop iterating over columns and the inner loop over rows.
- Print the elements in row-major order using a single loop and a linear index.

Program 2. Write a C program that reads elements into a variable-length 2D array, copies them into a 1D array in row-major order, and displays both arrays.

2. Algorithm

Program 1

Step 1: START

Step 2: Read rows. While rows is less than or equal to 0, display a message and read rows again.

Step 3: Read cols. While cols is less than or equal to 0, display a message and read cols again.

Step 4: Declare the variable-length array a of size rows × cols.

Step 5: Repeat for j = 0 to cols – 1:

Repeat for i = 0 to rows – 1: read a[i][j].

Step 6: Repeat for k = 0 to rows × cols – 1:

Print a[k / cols][k % cols].

If (k + 1) is a multiple of cols, move to a new line.

Step 7: STOP

Program 2

Step 1: START

Step 2: Read rows and cols.

Step 3: Declare the variable-length 2D array a of size rows × cols and the variable-length 1D array b of size rows × cols.

Step 4: Repeat for i = 0 to rows – 1:

Repeat for j = 0 to cols – 1: read a[i][j].

Step 5: Set k = 0. Repeat for i = 0 to rows – 1:

Repeat for j = 0 to cols – 1: set b[k] = a[i][j] and increase k by 1.

Step 6: Display the 2D array row by row.

Step 7: Display the 1D array b from b[0] to b[rows × cols – 1].

Step 8: STOP

3. Test Case Table

#	Program	Input	Expected Output	Actual Output	Result
1	Program 1	2×3 , entered column by column as 1, 4, 2, 5, 3, 6	1 2 3 on the first line and 4 5 6 on the second	1 2 3 / 4 5 6	PASS
2	Program 1	rows = -2, then 0, then 2	The message is shown twice and 2 is accepted	Shown twice	PASS
3	Program 1	cols = -1, then 2	The message is shown once and 2 is accepted	Shown once	PASS
4	Program 1	2×2 with 10, 30, 20, 40 column by column	10 20 on the first line and 30 40 on the second	10 20 / 30 40	PASS
5	Program 2	3×2 with 1 to 6	The 2D array in 3 rows and the 1D array as 1 2 3 4 5 6	1 2 3 4 5 6	PASS
6	Program 2	1×4 with 7, 8, 9, 10	A single row, and the 1D array the same as that row	7 8 9 10	PASS

4. Source Code

Program 1 — rowmajor.c

/* Program 1. Reads the number of rows and columns, makes sure both are positive, declares a variable length 2D array of that size, reads the elements column by column, and then prints them in row-major order using one single loop with a linear index. */

```
#include <stdio.h>

int main(void)
{
    int rows, cols, i, j, k;

    printf("Enter the number of rows: ");
    scanf("%d", &rows);
    while (rows <= 0) {
        printf("Rows must be positive. Enter again: ");
        scanf("%d", &rows);
    }

    printf("Enter the number of columns: ");
    scanf("%d", &cols);
    while (cols <= 0) {
        printf("Columns must be positive. Enter again: ");
        scanf("%d", &cols);
    }

    /* The size is known only now, at run time, so this is a variable
       length array. */
    int a[rows][cols];

    printf("Enter the elements column by column:\n");
    for (j = 0; j < cols; j++) {
        for (i = 0; i < rows; i++) {
            printf("a[%d][%d]: ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* One loop only. The linear index k runs from 0 to rows*cols-1, and
       k/cols gives the row while k%cols gives the column, which is
       exactly row-major order. */
    printf("\nElements in row-major order:\n");
    for (k = 0; k < rows * cols; k++) {
        printf("%d ", a[k / cols][k % cols]);
        if ((k + 1) % cols == 0)
            printf("\n");
    }

    return 0;
}
```

Program 2 — copy1d.c

/* Program 2. Reads the elements into a variable length 2D array, copies them into a 1D array in row-major order, and displays both arrays. */

```
#include <stdio.h>

int main(void)
{
    int rows, cols, i, j, k;

    printf("Enter the number of rows: ");
    scanf("%d", &rows);
    printf("Enter the number of columns: ");
    scanf("%d", &cols);
```

```

/* Both sizes are decided at run time, so both of these are
   variable length arrays. */
int a[rows][cols];
int b[rows * cols];

printf("Enter the elements row by row:\n");
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        printf("a[%d][%d]: ", i, j);
        scanf("%d", &a[i][j]);
    }
}

/* Row-major order means the first row is copied fully, then the
   second row, and so on. k just keeps counting forward. */
k = 0;
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        b[k] = a[i][j];
        k++;
    }
}

printf("\nThe 2D array:\n");
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++)
        printf("%d ", a[i][j]);
    printf("\n");
}

printf("\nThe 1D array in row-major order:\n");
for (k = 0; k < rows * cols; k++)
    printf("%d ", b[k]);
printf("\n");

return 0;
}

```

5. Compilation and Output

I compiled both files with `gcc -Wall -Wextra` so that even the small warnings would show up, and both compiled clean in the first attempt itself. The screenshots below are of the runs I did on my own machine.

Program 1, and the Repeated Input on Wrong Values

```
adilshakil — zsh — 127x34
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra rowmajor.c -o rowmajor
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra copyld.c -o copyld
adilshakil@ADILs-MacBook-Air MyAssignment % ./rowmajor
Enter the number of rows: 2
Enter the number of columns: 3
Enter the elements column by column:
a[0][0]: 1
a[1][0]: 4
a[0][1]: 2
a[1][1]: 5
a[0][2]: 3
a[1][2]: 6

Elements in row-major order:
1 2 3
4 5 6
adilshakil@ADILs-MacBook-Air MyAssignment % ./rowmajor
Enter the number of rows: -2
Rows must be positive. Enter again: 0
Rows must be positive. Enter again: 2
Enter the number of columns: -1
Columns must be positive. Enter again: 2
Enter the elements column by column:
a[0][0]: 10
a[1][0]: 30
a[0][1]: 20
a[1][1]: 40

Elements in row-major order:
10 20
30 40
█
```

Program 2

```
adilshakil — zsh — 127x34
adilshakil@ADILs-MacBook-Air MyAssignment % ./copyld
Enter the number of rows: 3
Enter the number of columns: 2
Enter the elements row by row:
a[0][0]: 1
a[0][1]: 2
a[1][0]: 3
a[1][1]: 4
a[2][0]: 5
a[2][1]: 6

The 2D array:
1 2
3 4
5 6

The 1D array in row-major order:
1 2 3 4 5 6
adilshakil@ADILs-MacBook-Air MyAssignment % ./copyld
Enter the number of rows: 1
Enter the number of columns: 4
Enter the elements row by row:
a[0][0]: 7
a[0][1]: 8
a[0][2]: 9
a[0][3]: 10

The 2D array:
7 8 9 10

The 1D array in row-major order:
7 8 9 10
█
```

All six test cases came out right. In the first run of Program 1 I typed the numbers in the order 1, 4, 2, 5, 3, 6 because the reading goes down the first column and then down the second, and the program still printed them as 1 2 3 and then 4 5 6, which is the row-major order the question asks for.

The single-loop printing is the part I had to think about. There is only one index k , running from 0 to $\text{rows} \times \text{cols} - 1$, and the row and the column are worked out from it as k / cols and $k \% \text{cols}$. For a 2×3 array $k = 4$ gives row 1 and column 1, which is the element 5, and that is exactly where 5 sits in the output.